Sprawozdanie z projektu nr 1 - PEA

Asymetryczny Problem Komiwojażera (ATSP)

Twoje Imię i Nazwisko / Nr albumu

31 marca 2026

Spis treści

1 Wstęp teoretyczny **2**

1.1 Sformułowanie problemu komiwojażera (TSP i ATSP) 2

1.2 Analiza i złożoność implementowanych algorytmów 2

1.2.1 Przegląd zupełny (Brute-Force) 3

1.2.2 Algorytm losowy (Random Search) 3

1.2.3 Najbliższy sąsiad (Nearest Neighbour - NN) 3

1.2.4 Powtarzalny najbliższy sąsiad (Repetitive Nearest Neighbour - RNN) 4

2 Przykład praktyczny algorytmu Nearest Neighbour dla $N = 3$ **4**

3 Opis implementacji algorytmów i zastosowanych struktur **6**

3.1 Kluczowe struktury danych i mechanizmy pomocnicze 6

3.2 Implementacja przeglądu zupełnego (Brute-Force) 7

3.3 Implementacja algorytmu losowego 7

3.4 Implementacja algorytmów zachłannych (NN oraz RNN) 8

4 Plan i metodologia eksperymentów **8**

1 Wstęp teoretyczny

1.1 Sformułowanie problemu komiwojażera (TSP i ATSP)

Problem komiwojażera (ang. *Traveling Salesperson Problem*, TSP) to jedno z najważniejszych zagadnień optymalizacji kombinatorycznej oraz informatyki teoretycznej. W swojej klasycznej postaci problem ten można sformułować następująco: dany jest zbiór N miast oraz odległości pomiędzy każdą parą z nich. Celem jest znalezienie najkrótszej możliwej trasy, która rozpoczyna się w dowolnym mieście, odwiedza dokładnie raz każde z pozostałych miast i powraca do miasta początkowego[cite: 3].

W ujęciu teorii grafów, problem modelowany jest za pomocą pełnego grafu ważonego $G = (V, E)$, gdzie V to zbiór wierzchołków (miast) o mocy $|V| = N$, a E to zbiór krawędzi (połączeń między miastami). Każdej krawędzi $(i, j) \in E$ przypisana jest waga (koszt) $c_{ij} \geq 0$. Szukanym rozwiązaniem jest cykl Hamiltona o minimalnej sumie wag krawędzi.

W niniejszym projekcie rozpatrywany jest wariant asymetryczny problemu komiwojażera (ang. *Asymmetric Traveling Salesperson Problem*, ATSP)[cite: 3]. Oznacza to, że graf G jest grafem skierowanym, a macierz kosztów przejść nie musi być symetryczna, czyli w ogólnym przypadku zachodzi:

$$c_{ij} \neq c_{ji} \quad \text{dla} \quad i, j \in V$$

Asymetria ta sprawia, że problem jest jeszcze trudniejszy do rozwiązania w optymalnym czasie. Ścieżka optymalna w jedną stronę może mieć zupełnie inny koszt (lub być wręcz niemożliwa ze względu na koszty dążące do nieskończoności) przy próbie jej odwrócenia. Problem ATSP należy do klasy problemów NP-trudnych, co oznacza, że nie jest znany żaden algorytm zdolny do znalezienia optymalnego rozwiązania w czasie wielomianowym dla każdego przypadku testowego.

1.2 Analiza i złożoność implementowanych algorytmów

W ramach prac projektowych zaimplementowano i zbadano cztery zróżnicowane podejścia algorytmiczne do problemu ATSP[cite: 3]:

1.2.1 Przegląd zupełny (Brute-Force)

Algorytm przeglądu zupełnego to metoda dokładna, gwarantująca odnalezienie globalnego optimum. Jego działanie polega na wygenerowaniu przestrzeni wszystkich możliwych rozwiązań (cykli Hamiltona), obliczeniu funkcji celu (kosztu całkowitego) dla każdej z nich i wybraniu wartości minimalnej.

Z uwagi na fakt, że poszukiwany jest cykl, punkt startowy nie ma znaczenia dla ostatecznego kosztu całej trasy. Dlatego w zaimplementowanym algorytmie wierzchołek początkowy (miasto o indeksie 0) jest na stałe „zamrożony" na pierwszej pozycji. Pozwala to na zredukowanie liczby permutacji do wygenerowania z $N!$ do $(N-1)!$. Całkowita złożoność obliczeniowa tego podejścia wynosi $O(N!)$. Złożoność silniowa rośnie w tak drastycznym tempie, że już dla stosunkowo niewielkich wartości N (np. $N > 13$) czas potrzebny na wygenerowanie i sprawdzenie wszystkich tras przekracza możliwości współczesnych jednostek obliczeniowych w akceptowalnym czasie ludzkim.

1.2.2 Algorytm losowy (Random Search)

Algorytm losowy należy do grupy metod stochastycznych (probabilistycznych). Zamiast systematycznie przeszukiwać przestrzeń rozwiązań, algorytm ten w sposób ciągły losuje permutacje ścieżek i oblicza ich koszty, zachowując w pamięci tę, która do tej pory charakteryzowała się najmniejszą wagą sumaryczną. Nie jest to algorytm heurystyczny, gdyż nie wykorzystuje żadnej logiki problemu do kierowania swoimi decyzjami – opiera się wyłącznie na ślepym losowaniu przestrzeni.

Pojedyncza iteracja (wylosowanie permutacji i obliczenie kosztu trasy o długości N) kosztuje asymptotycznie $O(N)$. Całkowita złożoność jest ściśle uzależniona od warunku stopu, którym w tym przypadku jest nałożony limit czasu[cite: 10]. Choć algorytm ten wykonuje ogromną liczbę prób w jednostce czasu, prawdopodobieństwo losowego „trafienia" w optymalną trasę przy dużej liczbie wierzchołków jest bliskie zeru. Algorytm ten często stanowi dolną granicę jakości (ang. *baseline*) w ocenie pozostałych metod.

1.2.3 Najbliższy sąsiad (Nearest Neighbour - NN)

Jest to klasyczny algorytm zachłanny[cite: 3]. Konstrukcja rozwiązania polega na wyborze wierzchołka startowego, a następnie, w każdym kolejnym kroku, dołączaniu do ścieżki tego spośród nieodwiedzonych wierzchołków, do którego koszt przejścia z obecnego węzła jest najmniejszy (lokalne optimum).

Decyzje podejmowane w każdym kroku są ostateczne (algorytm nie posiada mechanizmu nawrotów – ang. *backtracking*). Z tego powodu algorytm NN często wpada w tzw. pułapki – zmuszony jest na samym końcu trasy do wybrania wyjątkowo kosztownych krawędzi zamykających cykl, co drastycznie pogarsza całkowity wynik. Złożoność obliczeniowa to $O(N^2)$, ponieważ dla każdego z N miast algorytm musi przeszukać listę do $N-1$ potencjalnych sąsiadów. Jest to metoda skrajnie szybka, generująca rozwiązania natychmiastowo nawet dla bardzo dużych instancji problemu.

1.2.4 Powtarzalny najbliższy sąsiad (Repetitive Nearest Neighbour - RNN)

Jakość rozwiązania zwracanego przez zwykły algorytm NN jest silnie uzależniona od wyboru wierzchołka startowego – szczególnie w wariancie asymetrycznym macierzy. Aby zminimalizować to ryzyko, algorytm RNN uruchamia heurystykę Nearest Neighbour N razy, za każdym razem inicjalizując trasę w innym wierzchołku[cite: 3]. Po zebraniu N różnych cykli, algorytm zwraca ten o najniższym koszcie. Złożoność czasowa wzrasta o jeden rząd wielkości do $O(N^3)$, co nadal plasuje go w kategorii algorytmów wielomianowych o bardzo wysokiej wydajności praktycznej.

2 Przykład praktyczny algorytmu Nearest Neighbour dla $N = 3$

Dla lepszego zobrazowania mechaniki algorytmów zachłannych w przestrzeni asymetrycznej, przeanalizujmy działanie algorytmu Nearest Neighbour (NN) krok po kroku dla mikroskopijnej instancji problemu o rozmiarze $N = 3$. Niech asymetryczna macierz kosztów przejść M prezentuje się następująco:

$$M = \begin{bmatrix} \infty & 10 & 15 \\ 5 & \infty & 9 \\ 6 & 12 & \infty \end{bmatrix}$$

Wartość $M[i][j]$ określa koszt jednokierunkowej podróży z miasta i do miasta j. Na przekątnej znajdują się wartości ∞, uniemożliwiające podróż z danego miasta do niego samego.

Założenia początkowe: Algorytm wymusza miasto startowe na indeksie 0.

Krok 1: Stan początkowy

- Aktualne miasto: 0.
- Zbiór miast odwiedzonych: $V_{odwiedz} = \{0\}$.
- Zbiór miast nieodwiedzonych: $V_{nieodwiedz} = \{1, 2\}$.
- Aktualna ścieżka: $P = [0]$. Aktualny koszt całkowity: $C = 0$.

Krok 2: Poszukiwanie pierwszego sąsiada

- Z miasta 0 algorytm skanuje krawędzie prowadzące do wierzchołków z $V_{nieodwiedz}$.
- Koszt podróży do miasta 1: $M[0][1] = 10$.
- Koszt podróży do miasta 2: $M[0][2] = 15$.
- Zgodnie z kryterium zachłannym, algorytm wybiera minimum lokalne: $\min(10, 15) = 10$. Wybrane zostaje miasto 1.
- Aktualizacja stanu: $V_{odwiedz} = \{0, 1\}$, $V_{nieodwiedz} = \{2\}$.
- Aktualna ścieżka: $P = [0, 1]$. Aktualny koszt: $C = 0 + 10 = 10$.

Krok 3: Poszukiwanie drugiego sąsiada

- Z miasta 1 pozostał tylko jeden wierzchołek do odwiedzenia: zbiór $V_{nieodwiedz}$ zawiera jedynie element 2.
- Algorytm musi udać się do miasta 2. Koszt podróży to: $M[1][2] = 9$.
- Aktualizacja stanu: $V_{odwiedz} = \{0, 1, 2\}$, zbiór miast nieodwiedzonych jest pusty.
- Aktualna ścieżka: $P = [0, 1, 2]$. Aktualny koszt: $C = 10 + 9 = 19$.

Krok 4: Zamknięcie cyklu Hamiltona

- Aby poprawnie rozwiązać problem komiwojażera, trasę należy zakończyć powrotem z ostatniego miasta do wierzchołka startowego.
- Koszt powrotu z miasta 2 do miasta 0 wynosi: $M[2][0] = 6$.
- **Ostateczna ścieżka:** $P_{final} = [0, 1, 2, 0]$.
- **Ostateczny koszt całkowity:** $C_{final} = 19 + 6 = 25$.

3 Opis implementacji algorytmów i zastosowanych struktur

Całość projektu została zaimplementowana w języku C++ z rygorystycznym zachowaniem paradygmatu programowania obiektowego[cite: 11]. Aby spełnić założenia optymalnego zarządzania pamięcią, zrezygnowano ze statycznych tablic wielowymiarowych. Używane struktury danych są alokowane w pełni dynamicznie na stercie w zależności od żądanego rozmiaru badanej instancji N[cite: 5].

3.1 Kluczowe struktury danych i mechanizmy pomocnicze

- **Klasa `Array`:** Autorska, obiektowa reprezentacja asymetrycznej macierzy kosztów. Wewnątrz alokuje ciągły blok pamięci, co optymalizuje proces prefetchingu przez pamięć podręczną procesora i minimalizuje błędy braku w pamięci (ang. *cache misses*) podczas gęstego odpytywania w algorytmach o wysokiej złożoności. Udostępnia metody dostępowe takie jak `get(row, col)`.

- **Klasa `Wektor`:** Dynamiczna tablica wskaźnikowa służąca do przechowywania permutacji miast (ścieżek) oraz flag odwiedzin w algorytmach zachłannych. Implementuje własne mechanizmy kopiowania (konstruktor kopiujący, operator przypisania) zapobiegające wyciekom pamięci.

- **Struktura `Result`:** Pełni rolę kontenera transportowego dla wyników działania każdego z algorytmów. Posiada pole `cost` reprezentujące minimalną znalezioną odległość oraz obiekt `path` (typu `Wektor`) przechowujący poprawną kolejność wierzchołków.

- **Zarządzanie czasem wykonania:** Wykorzystano wprowadzony w standardzie C++20 mechanizm `std::stop_token`. Umożliwia on eleganckie i przede wszystkim w pełni bezpieczne z punktu widzenia wielowątkowości przerywanie działania nieskończonych pętli (jak w metodzie `tsp_random`) lub pętli o potężnej liczbie iteracji (jak w metodzie `tsp_brute_force`) przez zewnętrzny kontroler czasu, eliminując potrzebę inwazyjnego odpytywania funkcji systemowych o aktualny czas wewnątrz najbardziej wrażliwych bloków obliczeniowych.

3.2 Implementacja przeglądu zupełnego (Brute-Force)

Sercem algorytmu dokladnego jest funkcja `tsp_brute_force`. Pętla poszukująca optymalnego rozwiązania została oparta o konstrukcję `do-while`. Fundamentalnym problemem optymalizacyjnym jest tu generowanie permutacji. Zrezygnowano z bibliotecznej funkcji `std::next_permutation` na rzecz implementacji własnej metody `nastepna_permutacja` operującej na autorskiej klasie `Wektor`.

Algorytm generowania permutacji opiera się na porządku leksykograficznym i składa się z następujących kroków:

1. Poszukiwanie od końca (od prawej do lewej) pierwszego takiego indeksu i, dla którego zachodzi $w[i] < w[i+1]$. Jeśli taki indeks nie istnieje, wygenerowano już wszystkie możliwe układy.
2. Poszukiwanie od końca pierwszego indeksu j, dla którego $w[j] > w[i]$.
3. Zamiana wartości pod indeksami i oraz j (`std::swap`).
4. Odwrócenie kolejności elementów na prawo od indeksu i (od $i + 1$ do końca) za pomocą pomocniczej metody `odworoc_wektor`.

Co niezwykle istotne, implementacja ta ignoruje indeks 0 wektora ścieżki. Złożoność $(N - 1)!$ uzyskuje się poprzez zablokowanie pierwszego miasta i permutowanie zbioru wierzchołków od indeksu 1 do $N - 1$.

3.3 Implementacja algorytmu losowego

Funkcja `tsp_random` ma za zadanie działać nieprzerwanie aż do momentu zgłoszenia żądania zatrzymania przez `std::stop_token`. Za generowanie pseudolosowości odpowiada wysoce wydajny generator Mersenne Twister (`std::mt19937`), inicjalizowany za pomocą ziarna pochodzącego z entropii sprzętowej systemu (`std::random_device`).

Do tasowania miast wykorzystano klasyczny i optymalny algorytm Fishera-Yatesa, zaimplementowany wewnątrz głównej pętli `while(true)`. Złożoność pojedynczego przetasowania wynosi $O(N)$. Podobnie jak w przeglądzie zupełnym, tasowaniu podlega wyłączone sub-pasmo wektora od indeksu 1 do $N-1$, ponieważ miasto startowe musi pozostać niezmienione. Po każdym przetasowaniu następuje ewaluacja wagi grafu i ewentualne nadpisanie struktury `out_result`.

3.4 Implementacja algorytmów zachłannych (NN oraz RNN)

Główna logika Nearest Neighbour znajduje się w wydzielonej metodzie `tsp_nn_wew`. Stan algorytmu śledzony jest przy użyciu wektora `visited`, zainicjowanego zerami. Algorytm wymusza $N - 1$ kroków iteracyjnych. W każdym z nich wewnętrzna pętla analizuje wszystkie N kolumn macierzy kosztów. Instrukcja warunkowa `if (visited[i] == 0)` stanowi swoisty filtr, dzięki któremu obliczenia i weryfikacja minimum (`dist < min_dist`) odbywają się wyłącznie dla wierzchołków dotychczas nieeksplorowanych. Złożona w ten sposób trasa wytycza ścieżkę do przedostatniego węzła. Wartość kosztu uzupełniana jest o wagę krawędzi powrotnej na zewnątrz głównej pętli iteracyjnej.

Rozszerzenie RNN (`tsp_rnn`) implementuje pętlę inkrementującą parametr `start_city` od 0 do $N - 1$. W każdej iteracji stan wewnętrzny odwiedzin jest resetowany i następuje kompletne uruchomienie logiki Nearest Neighbour opierające się na wybranym `start_city`. Wyniki poszczególnych wywołań są agregowane, a program zwraca globalne minimum z tego konkretnego zbioru prób.

4 Plan i metodologia eksperymentów

Aby ocenić skuteczność, poprawność oraz efektywność czasową zaimplementowanych algorytmów w sposób empiryczny i rygorystyczny, zaplanowano następujący scenariusz badawczy:

1. Konfiguracja środowiska testowego i mechanika pomiarów:
Wszystkie testy wykonane zostaną na spójnym środowisku sprzętowym, aby zagwarantować powtarzalność i porównywalność wyników. W środowisku programistycznym Qt Creator program przed wykonaniem testów wydajnościowych zostanie skompilowany w docelowej konfiguracji **Release**. Krok ten jest kluczowy do uruchomienia flag i optymalizacji kompilatora, co gwarantuje miarodajność wyników pomiaru czasu dla złożonych obliczeń[cite: 35]. Pomiary będą dokonywane z wykorzystaniem biblioteki `<chrono>` za pomocą wysokiej rozdzielczości zegara monotonicznego `high_resolution_clock`.

2. Struktura danych wejściowych i próbkowanie uśredniające:
Pomiary przeprowadzone zostaną dla instancji problemów wczytywanych z plików tekstowych oraz losowo generowanych instancji[cite: 6]. Zgodnie z wymaganiami projektowymi, testy obejmą minimum 7 zróżnicowanych, reprezentatywnych wielkości problemu N[cite:

7]. Z uwagi na radykalnie różne klasy złożoności, wartości N będą dobierane osobno:

- **Dla Brute-Force:** Oczekiwane skale problemu to wartości niewielkie, na których funkcja silni wykazuje dramatyczny przyrost czasu, np. $N \in \{5, 8, 10, 11, 12, 13, 14\}$.
- **Dla algorytmów przybliżonych (RAND, NN, RNN):** Ponieważ są to metody z czasem wielomianowym bądź stochastycznym, pomiary czasu przyjmą wielokrotnie wyższe pułapy instancji wejściowych, np. $N \in \{10, 50, 100, 250, 500, 750, 1000, 2500\}$.

W celu zniwelowania błędu pomiarowego wynikającego z ingerencji systemu operacyjnego, dla danego rozmiaru N test wydajnościowy zostanie zwielokrotniony (wykonany dla różnych wygenerowanych map o danym rozmiarze lub w pętli dla jednej instancji), a do finalnego sprawozdania trafią wyłącznie wartości uśrednione[cite: 7].

3. Poszukiwanie granicy wykonania w ‚rozsądnym czasie":
Ważnym punktem eksperymentu będzie wyznaczenie dla każdej z czterech metod maksymalnego rozmiaru instancji grafu wejściowego N_{max}, dla którego program jest w stanie zwrócić kompletne rozwiązanie w założonym „rozsądnym czasie", który zdefiniowano na przedział od 10 do 30 minut[cite: 8, 9]. Doświadczenie to ukaże barierę skalowalności metod dokładnych.

4. Ewaluacja błędu względnego rozwiązań aproksymowanych:
Z racji faktu, iż algorytmy RAND, NN oraz RNN nie gwarantują odnalezienia rozwiązania najkrótszego w grafie, obliczona zostanie metryka błędu względnego δ[cite: 10]. Wzór wykorzystywany w analizie jakościowej:

$$\delta = \frac{C_{heur} - C_{opt}}{C_{opt}} \cdot 100\%$$

gdzie C_{heur} to całkowity koszt ścieżki wyznaczony przez dany algorytm przybliżony, natomiast C_{opt} to globalne optymalne rozwiązanie (pozyskane z analizy dokonanej za pomocą przeglądu zupełnego lub podane jako pewnik pochodzący z ogólnodostępnych baz).

5. Analiza parametryczna algorytmów:
W przypadku algorytmu w pełni losowego (RAND), zbadana zostanie krzywa poprawy jakości wyznaczonej ścieżki (spadek błędu względnego δ) w funkcji zadanego z góry reżimu czasowego[cite: 10]. Analiza odpowie na pytanie, czy dwukrotne lub dziesięciokrotne zwiększenie budżetu czasowego wpłynie na optymalizację trasy. Wyniki eksperymentów

zostaną zwizualizowane za pomocą dedykowanych wykresów liniowych i słupkowych[cite: 17].